13 - Decision Trees and Random Forest

Artificial Intelligence Lab Manual - Clean Study Edition

Learning Objectives

- Explain recursive feature-based splitting in decision trees.
- Train and evaluate a decision tree classifier.
- Explain how a random forest combines many randomized trees to improve generalization.

Background

Decision trees form human-readable if-then structures by recursively selecting informative splits. Deep trees can fit noise. Random forests reduce variance by averaging predictions from many diverse trees, usually improving robustness at the cost of simple single-tree interpretability.

Core Concepts

Concept	Meaning in this lab
Split	Rule such as feature <= threshold that partitions examples.
Impurity	Measure such as Gini impurity or entropy used to score candidate splits.
Leaf	Terminal node that outputs a prediction.
Random forest	Ensemble of trees trained on resampled data and randomized feature subsets.
Overfitting control	Depth limits, minimum samples, pruning, and ensemble averaging.

Guided Procedure

1. Load a binary or multiclass tabular dataset and separate features from the target.

2. Create train and test partitions.

3. Train a shallow decision tree and record accuracy, depth, and feature importances.

4. Train an unrestricted tree and compare test performance for signs of overfitting.

5. Train a random forest and compare accuracy and stability with the single tree.

Reference Implementation

Use the following as a starting point. Read and modify it rather than treating it as a final answer.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

tree = DecisionTreeClassifier(max_depth=4, random_state=7)
forest = RandomForestClassifier(
    n_estimators=200, max_depth=None, random_state=7
)

tree.fit(X_train, y_train)
forest.fit(X_train, y_train)

print(tree.score(X_test, y_test))
print(forest.score(X_test, y_test))
```

Lab Exercises

Task 1. Compare Gini impurity and entropy on the same dataset.

Task 2. Vary max_depth and plot training versus test accuracy.

Task 3. Train random forests with 10, 50, 100, and 200 trees and compare results.

Task 4. Rank features by random-forest importance and discuss whether importance implies causation.

Suggested Deliverables

- Notebook or Python file containing the completed implementation.
- Short result table or output evidence for each required comparison.
- A concise interpretation of what changed and why; do not submit output without explanation.

Review Questions

1. Why can deep decision trees overfit?

2. How does a random forest create diversity among trees?

3. Why is a single tree easier to interpret than a forest?

Completion check: You should be able to explain the algorithm or workflow in your own words, reproduce the main result, and identify at least one limitation.